# Interferencia cuántica

Genesis Silesky

20 de octubre de 2026

## 1. Interferencia cuántica

### 1.1. Objetivo

Mostrar que las amplitudes se suman y pueden cancelarse, y que por eso el resultado de un circuito depende de las *fases* y no solo de las probabilidades intermedias.

### 1.2. Conceptos previos

**Concepto previo: amplitudes con signo y suma de amplitudes**

Ver sección **??**. Una amplitud puede ser negativa o compleja; una probabilidad no. Al encadenar puertas, las amplitudes que llegan al mismo resultado se *suman* (con su signo/fase) *antes* de elevar al cuadrado.

### 1.3. Fundamento teórico

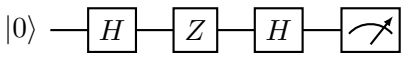
Aplicando $H$ dos veces a $|0\rangle$:

$$H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \tag{1}$$

$$HH|0\rangle = \tfrac{1}{\sqrt{2}}\Big[\tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\Big] = |0\rangle\,. \tag{2}$$

La amplitud de $|1\rangle$ es $\frac{1}{2} - \frac{1}{2} = 0$ (interferencia *destructiva*); la de $|0\rangle$ es $\frac{1}{2} + \frac{1}{2} = 1$ (*constructiva*). Un modelo clásico probabilístico (50/50 y luego 50/50) daría de nuevo 50/50, porque las probabilidades nunca se cancelan; las amplitudes sí.

Con una fase en medio, $Z$ cambia el signo de $|1\rangle$: $ZH|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$ y $H|-\rangle = |1\rangle$. En general $HZH = X$.




Este es el principio de los algoritmos cuánticos: diseñar fases para que las respuestas incorrectas se cancelen y la correcta se refuerce (capítulo 06).

### 1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 05a - Interferencia
# Las amplitudes se suman o se cancelan. Como H H = I, el |1> se cancela.

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
```

```python
from qiskit.quantum_info import Statevector

# %% Celda 2: una H frente a dos H
# Una sola H: superposicion 50/50.
una = QuantumCircuit(1)
una.h(0)
p1 = {str(k): round(float(v), 4) for k, v in Statevector(una).probabilities_dict().items()}
print("1 H :", Statevector(una).data.round(3), p1)

# Dos H seguidas: la segunda H combina las dos amplitudes de la primera.
# La contribucion hacia |1> es (+1/2) + (-1/2) = 0: interferencia
# DESTRUCTIVA. La contribucion hacia |0> es (+1/2) + (+1/2) = 1:
# interferencia CONSTRUCTIVA. El qubit vuelve a |0> con certeza.
dos = QuantumCircuit(1)
dos.h(0)
dos.h(0)
p2 = {str(k): round(float(v), 4) for k, v in Statevector(dos).probabilities_dict().items()}
print("2 H :", Statevector(dos).data.round(3), p2)

# %% Celda 3: la fase decide hacia donde interfiere
# La puerta Z cambia el signo de la amplitud de |1> (fase de pi).
# Entre dos H, ese signo invierte la interferencia: H Z H = X,
# asi que el resultado pasa de |0> a |1> con certeza.
fase = QuantumCircuit(1, 1)
fase.h(0)
fase.z(0)
fase.h(0)
fase.measure(0, 0)
# Esperado: {'1': 1000}. Solo cambiamos una fase y cambio el resultado,
# aunque la Z "no hace nada" a las probabilidades por si sola.
print(AerSimulator().run(fase, shots=1000).result().get_counts())
```

### 1.5. Explicación celda por celda

**Celda 2** Compara un circuito con una $H$ (50/50) y otro con dos $H$ (solo $|0\rangle$). Se usa `Statevector` para ver las amplitudes exactas, sin aleatoriedad.

**Celda 3** Inserta $Z$ entre las dos $H$ y mide 1000 shots: sale siempre "1". Solo se cambió una fase.

### 1.6. Salida obtenida

```
1 H : [0.707+0.j 0.707+0.j] {'0': 0.5, '1': 0.5}
2 H : [1.+0.j 0.+0.j] {'0': 1.0}
{'1': 1000}
```

Es determinista porque la medición final tiene probabilidad 1.

### 1.7. Interpretación

La puerta $Z$ "no hace nada" a las probabilidades de $|+\rangle$, pero cambia por completo el resultado final. Eso solo es posible porque la información de fase se almacena en el signo de las amplitudes.

### 1.8. Ejercicios

1. Reemplace $Z$ por la puerta $S$ (fase $i$) y prediga el resultado antes de ejecutarlo.
2. Pruebe $H\,X\,H$: ¿a qué puerta equivale?
3. Explique por qué un "coin flip" clásico repetido no reproduce el resultado de $HH$.